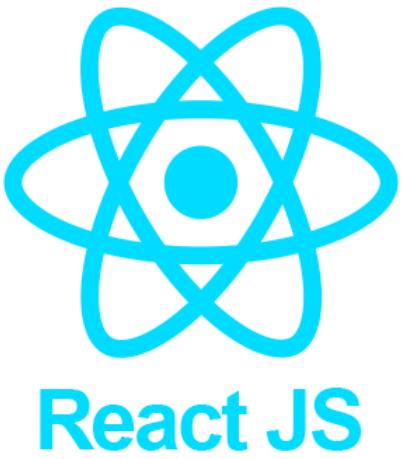
ESCUELA POLITÉCNICA NACIONAL
ESCUELA DE FORMACIÓN DE TECNÓLOGOS

DESARROLLO DE APLICACIONES WEB

PROFESOR:	Ing. Lorena Chulde
PERÍODO ACADÉMICO:	2026-B
FECHA DE ENTREGA:	10 – 11 - 2026

TALLER
(individual)

TÍTULO:
FRONTEND
REACT



Estudiante
Daniel Vásquez

PROPÓSITO DE LA PRÁCTICA

Desarrollar una interfaz web con React, Vite, JavaScript y Axios que consuma los endpoints REST del backend del Sistema de Ventas y permita realizar operaciones CRUD sobre los productos.

OBJETIVOS

Desarrollar el Frontend del Sistema de Ventas usando React + Vite + JavaScript (JSX).

Integrar frontend y backend.

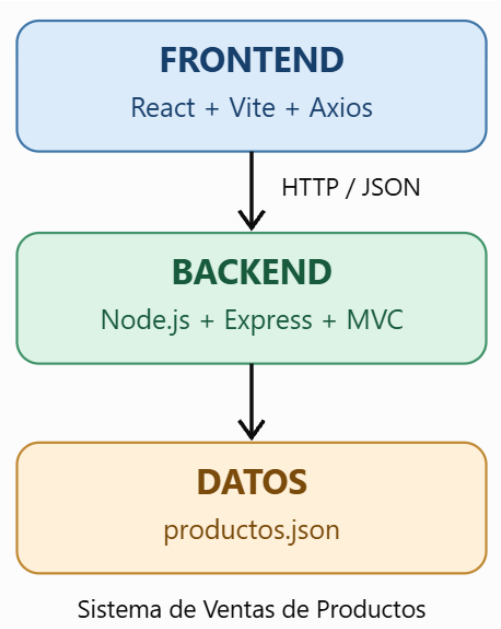
Recordar a cerca de componentes, estados, eventos y consumo de APIs REST.

Resultados de aprendizaje

Al finalizar la práctica, el estudiante podrá:

1. Explicar qué son React, Vite, JSX, componentes y Axios.
2. Crear y ejecutar un proyecto React utilizando Vite.
3. Organizar los archivos y componentes del frontend.
4. Consumir una API REST mediante Axios.
5. Utilizar useState y useEffect para gestionar datos.
6. Implementar operaciones CRUD desde una interfaz web.
7. Integrar el frontend con el backend desarrollado anteriormente.

Arquitectura de la aplicación



El frontend enviará solicitudes HTTP al backend utilizando Axios. El backend procesará esas solicitudes mediante las rutas, controladores y modelos que ya implementamos.

Los endpoints son:

Método	Endpoint	Operación
GET	/api/productos	Listar productos
GET	/api/productos/:id	Buscar producto
POST	/api/productos	Registrar producto
PUT	/api/productos/:id	Modificar producto
DELETE	/api/productos/:id	Eliminar producto

Tecnología utilizada

React

Biblioteca de JavaScript para construir interfaces de usuario mediante componentes reutilizables. Por ejemplo, podemos crear un componente para mostrar cada producto.

https://react.dev/learn

Vite

Herramienta que permite crear y ejecutar proyectos frontend, ofreciendo un servidor de desarrollo rápido y herramientas para preparar la aplicación para producción.

https://vite.dev/guide/

JavaScript y JSX

JavaScript implementa la lógica de la aplicación. JSX es una extensión de sintaxis que permite describir la interfaz utilizando etiquetas similares a HTML dentro del código JavaScript.

https://react.dev/learn/writing-markup-with-jsx

Axios

Biblioteca cliente HTTP basada en promesas. Nos permitirá realizar solicitudes GET, POST, PUT y DELETE desde React hacia nuestra API Express.

https://axios.rest/pages/getting-started/first-steps

Actividades: Frontend con React + Vite + JavaScript + Axios

N.º	Actividad	Contenidos
1	Crear y ejecutar el proyecto React con Vite	Instalación, estructura y primera interfaz.
2	Comprender JSX y componentes	Construir componentes de la interfaz.
3	Instalar y configurar Axios	Conexión con la API Express y CORS.
4	Consultar y mostrar productos	GET, useState, useEffect y map().
5	Registrar productos	Formulario, eventos y POST.
6	Modificar productos	Carga de datos y PUT.
7	Eliminar productos	Eventos, confirmación y DELETE.
8	Integración y pruebas finales	Validaciones y manejo de errores.

Parte 1. Crear el proyecto React con Vite

Crear, ejecutar y comprobar el funcionamiento inicial del frontend.

1. Verificar Node.js y npm

Abrir Visual Studio Code y ejecutar en la terminal:

```
node -v
npm -v
```

¿Qué hace cada herramienta?

Node es el motor que hace que el código corra; npm es la herramienta que descarga las piezas que necesitas para construirlo

La documentación actual de Vite requiere Node.js 20.19+ o 22.12+ (o una versión posterior compatible).

2. Crear el proyecto

Crear el directorio para el frontend y ejecutar:

```
npm create vite@latest frontend-ventas -- --template react
```

Este comando crea el proyecto **frontend-ventas** utilizando la plantilla de React con JavaScript.

Si aparece una pregunta para instalar **create-vite,** confirmar. Si aparecen opciones adicionales de configuración, mantener la plantilla React con JavaScript.

Selecciona ESLint.

ESLint: Detecta posibles errores y problemas en el código JavaScript y React.

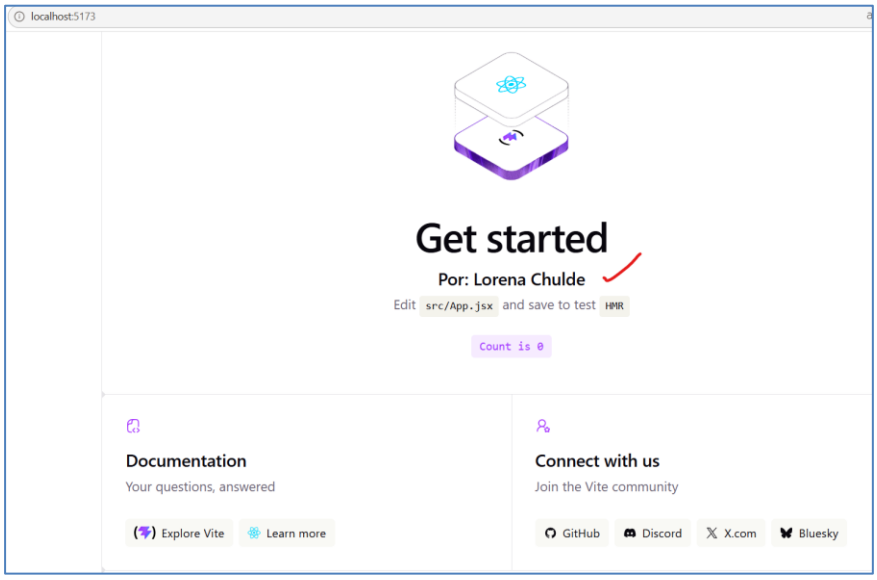
Oxlint: También analiza el código, pero está enfocado en ofrecer mayor velocidad.

3. Ejecutar el servidor de desarrollo

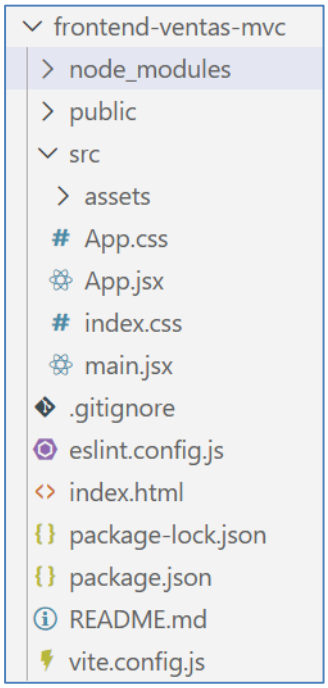
npm run dev

La terminal mostrará una dirección local, normalmente: **http://localhost:5173/**

Abrirla en el navegador. Si el puerto está ocupado, Vite puede utilizar otro.



4. Reconocer la estructura del proyecto



Archivo o directorio	Función
src/	Directorio principal que contiene todo el código fuente de la aplicación React
App.jsx	Componente principal de la aplicación que integra el diseño, el estado, los formularios y la lógica de negocio
main.jsx	Punto de entrada de la aplicación que monta el componente App dentro del elemento raíz del archivo HTML
App.css	Archivo de hojas de estilo específico para diseñar los componentes y la estructura visual de la aplicación
index.css	Archivo de estilos globales que define la base estética y tipográfica para todo el proyecto
package.json	Archivo de configuración que gestiona las dependencias, scripts de ejecución y metadatos del proyecto
vite.config.js	Archivo de configuración del empaquetador y servidor de desarrollo Vite

5. Crear la primera interfaz

Abrir **src/App.jsx,** eliminar su contenido y reemplazarlo por:

```jsx
function App() {
  return (
    <div>
      <h1>Sistema de Ventas</h1>
      <h2>Gestión de Productos</h2>
      <p>Frontend desarrollado con React y Vite.</p>
    </div>
  );
}

export default App;
```

React utiliza componentes que son funciones JavaScript y devuelven JSX.

Comprobar que la página cambia automáticamente.



Actividad: Personalizar la interfaz inicial.



Deberá modificar el componente **App.jsx** para incluir el nombre de su sistema, una descripción, mensaje de bienvenida y realizado por: “su nombre”. También deberá identificar qué archivo contiene el componente principal y comprobar que los cambios aparecen en el navegador sin reiniciar manualmente el servidor.

6. Preguntas de comprensión

1. ¿Cuál es la función de React en nuestra aplicación?

Construir la interfaz de usuario del lado del cliente mediante componentes reutilizables, reactivos y basados en un DOM virtual eficiente.

2. ¿Qué función cumple Vite?

Funciona como una herramienta de construcción (*build tool*) y servidor de desarrollo ultrarrápido que permite compilar, ejecutar y servir la aplicación frontend moderna con HMR (*Hot Module Replacement*)

3. ¿Qué diferencia existe entre JavaScript y JSX?

JavaScript es el lenguaje de programación estándar que maneja toda la lógica, mientras que JSX es una extensión de sintaxis propia de React que permite escribir código similar a HTML directamente dentro de los archivos .js

4. ¿Qué hace el comando npm install?

Lee el archivo y descarga las librerías, dependencias y paquetes necesarios en la carpeta donde este alojado el proyecto

5. ¿Para qué sirve export default App?

Permite exportar por defecto el componente principal App para que pueda ser importado y montado en el punto de entrada raíz de la aplicación

Parte 2. JSX y componentes en React

Construir una primera interfaz para visualizar productos utilizando componentes, JSX, propiedades (**props)** y arreglos de JavaScript.

Todavía no utilizaremos Axios. Primero comprenderemos cómo React construye y muestra los elementos de una página web.

Desarrollar componentes reutilizables en React para representar productos de un sistema de ventas, utilizando JSX, propiedades y estructuras de datos de JavaScript.

Al finalizar, los estudiantes podrán crear **componentes**, pasar información mediante **props** y mostrar varios productos utilizando **map().**

Conceptos fundamentales



¿Qué es un componente en React?

Es una pieza de código independiente y reutilizable que devuelve elementos visuales en formato JSX, representando una parte de la interfaz de usuario

Por ejemplo:

```
function Encabezado() {
  return <h1>Sistema de Ventas</h1>;
}
```

Podemos utilizarlo dentro de otro componente:

```
function App() {
  return (
    <div>
      <Encabezado />
    </div>
  );
}

export default App;
```

Documentación oficial:

https://react.dev/learn/your-first-component

¿Qué es JSX?

Es una extensión de la sintaxis de JavaScript que permite escribir código con una estructura muy similar a HTML o XML directamente dentro de archivos JavaScript

Ejemplo:

```
function Producto() {
  return <h2>{nombre}</h2>;
}
```

Las llaves **{ }** permiten incorporar expresiones de JavaScript dentro del JSX.

Algunas reglas importantes son que los elementos deben estar correctamente cerrados, el componente debe devolver un único elemento raíz y se utiliza **className** en lugar de **class** para las clases **CSS.**

Documentación oficial: React – Writing Markup with JSX.

https://react.dev/learn/writing-markup-with-jsx

¿Qué son las propiedades (props)?

Son los datos o parámetros que se le pasan a un componente para configurar su comportamiento o apariencia

```
function Producto({ nombre, precio }) {
  return (
    <div>
      <h3>{nombre}</h3>
      <p>Precio: ${precio}</p>
    </div>
  );
}
```

Uso:

```
<Producto nombre="Laptop Lenovo" precio={750} />
```

En este ejemplo, **nombre y precio** son propiedades que recibe el componente **Producto.**

Documentación oficial: React – Passing Props to a Component.

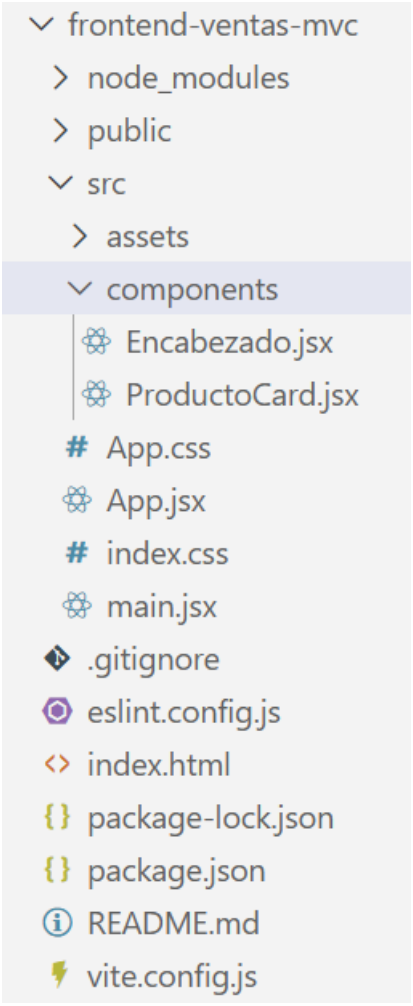
https://react.dev/learn/passing-props-to-a-component

Interfaz de productos

Organizar los componentes

Dentro del directorio **src,** crear una carpeta llamada **components.**

La estructura quedará de la siguiente manera:



¿Por qué es conveniente separar los componentes en archivos diferentes?

Es conveniente separar los componentes en archivos diferentes para mantener el código ordenado y fácil de encontrar, reutilizar las mismas piezas visuales en varias partes de la aplicación sin tener que repetir código, y facilitar las modificaciones o correcciones de errores en un solo lugar sin afectar al resto del sistema

Crear el componente Encabezado

Crear el archivo **src/components/Encabezado.jsx:**

```jsx
function Encabezado() {
  return (
    <header>
      <h1>Sistema de Ventas</h1>
      <p>Administración de productos</p>
    </header>
  );
}

export default Encabezado;
```

Este componente se encargará de mostrar el encabezado de nuestra aplicación.

Crear el componente ProductoCard

Crear el archivo **src/components/ProductoCard.jsx:**

```jsx
function ProductoCard({ nombre, precio, stock }) {
  return (
    <article className="producto-card">
      <h3>{nombre}</h3>
      <p>Precio: ${precio}</p>
      <p>Stock: {stock}</p>
    </article>
  );
}

export default ProductoCard;
```

Explicación del código:

Instrucción	Función
function ProductoCard(...)	Declara un componente funcional de React llamado ProductoCard encargado de renderizar la tarjeta visual de un producto.
{ nombre, precio, stock }	Desestructura las propiedades (*props*) que recibe el componente para extraer directamente los datos necesarios
<article>	Es una etiqueta HTML semántica que representa un contenedor independiente, utilizado aquí para estructurar la tarjeta del producto
className	Atributo de JSX equivalente a class en HTML, empleado para aplicar los estilos CSS a los elementos
{nombre}	Permite insertar y mostrar el valor de la variable JavaScript nombre directamente dentro del contenido visual de JSX usando llaves.
export default	Permite exportar el componente por defecto para que pueda ser importado y utilizado fácilmente en otros archivos

Crear un arreglo de productos

Por ahora utilizaremos datos locales, similares a los que tenemos en el archivo **productos.json** del backend.

Modificar **src/App.jsx:**

```jsx
import Encabezado from "./components/Encabezado";
import ProductoCard from "./components/ProductoCard";
import "./App.css";

function App() {
  const productos = [
    {
      id: 1,
      nombre: "Laptop Lenovo",
      precio: 750,
      stock: 10
    },
    {
      id: 2,
      nombre: "Mouse Logitech",
      precio: 25,
      stock: 30
    },
    {
      id: 3,
      nombre: "Teclado Redragon",
      precio: 45,
      stock: 15
    }
  ];
```

```
26
27      return (
28        <div className="contenedor">
29          <Encabezado />
30
31          <h2>Lista de productos</h2>
32
33          <div className="productos-grid">
34            {productos.map((producto) => (
35              <ProductoCard
36                key={producto.id}
37                nombre={producto.nombre}
38                precio={producto.precio}
39                stock={producto.stock}
40              />
41            ))}
42          </div>
43        </div>
44      );
45    }
46
47    export default App;
```

- **map()** transforma los elementos del arreglo en componentes.
- **producto** representa cada objeto del arreglo.
- **key={producto.id}** permite que React identifique cada elemento de la lista.
- Las demás propiedades envían los datos al componente.

Documentación oficial: React – Rendering Lists.

https://react.dev/learn/rendering-lists

Aplicar estilos CSS

Reemplazar el contenido de **src/App.css** por:

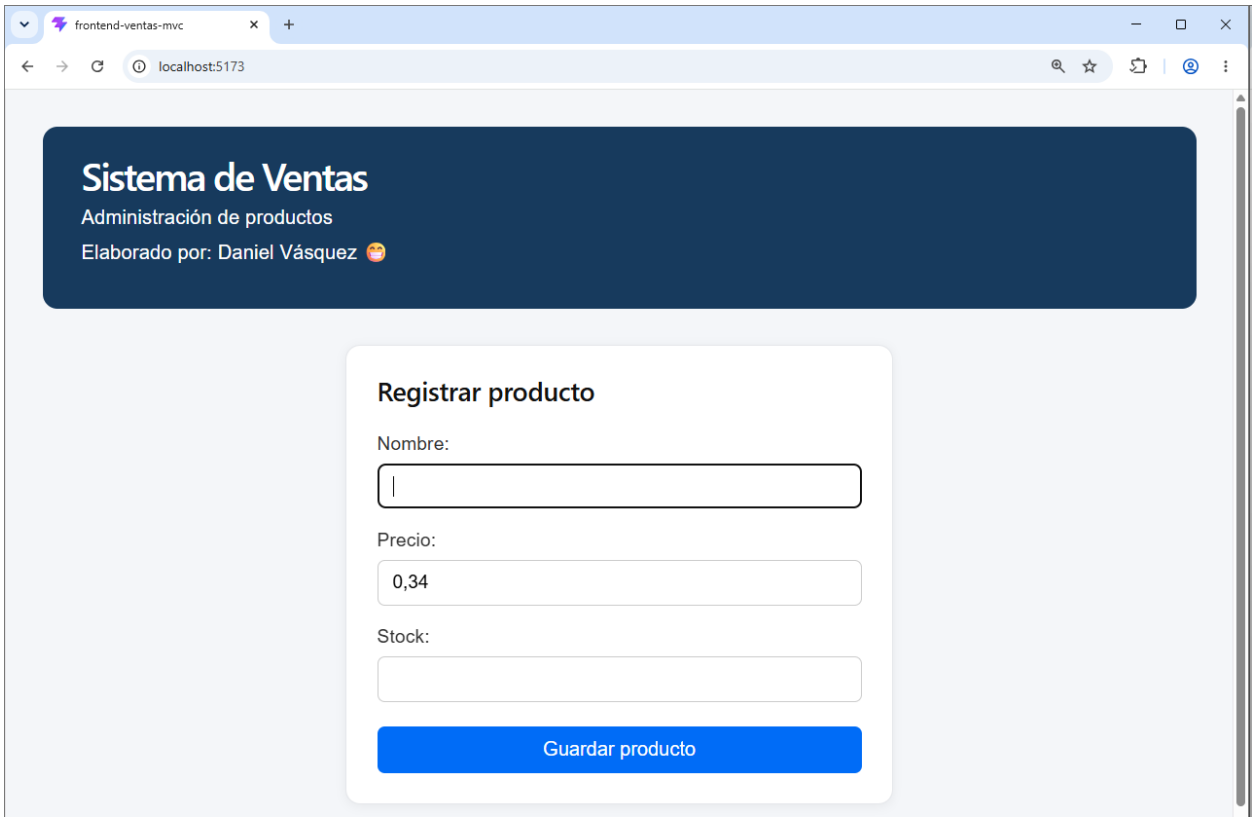
Descargarse los estilos desde OneDrive.

Para evitar que los estilos predeterminados de Vite alteren la distribución, reemplazar también **src/index.css:**

Descargarse los estilos desde OneDrive.

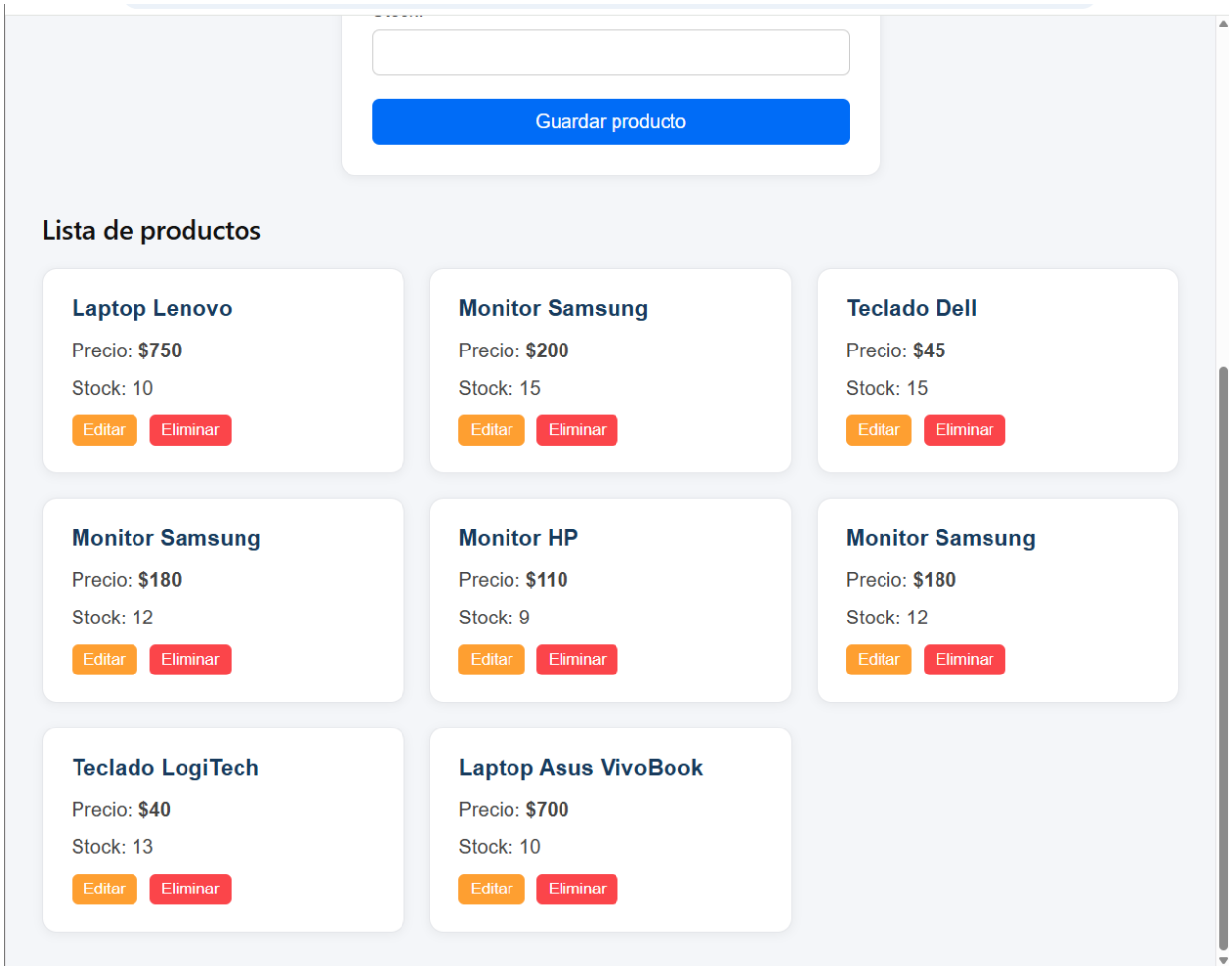
Resultado esperado

Al ejecutar **npm run dev,** la interfaz debe mostrar un encabezado y tres tarjetas de productos.



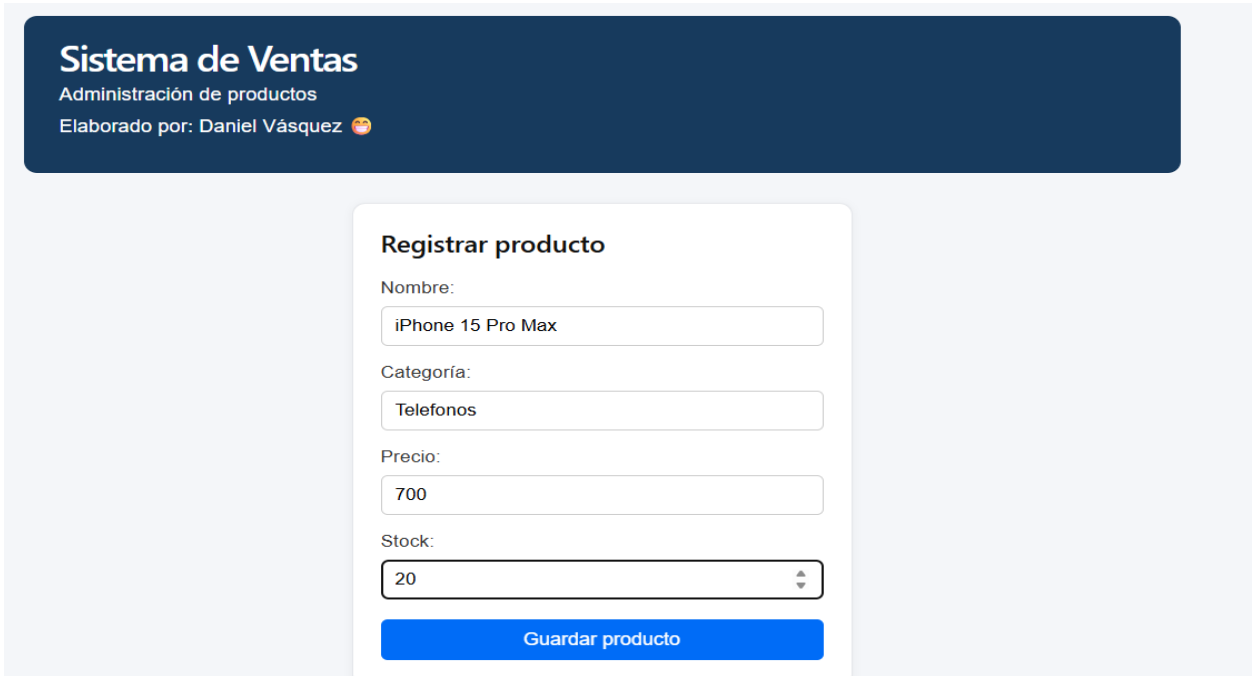
Actividad 1

Agregar dos productos nuevos al arreglo **productos,** cada uno con un ID diferente.



Actividad 2

Modificar **ProductoCard.jsx** para mostrar también la categoría del producto. Será necesario agregar la propiedad **categoria** en los objetos y enviarla desde **App.jsx.**



Actividad 3

Cambiar los estilos de las tarjetas para que el precio se muestre en negrita.

Preguntas de evaluación

1. ¿Qué es un componente en React?

Es una pieza de código independiente y reutilizable que devuelve elementos visuales en formato JSX, representando una parte de la interfaz de usuario

2. ¿Qué diferencia existe entre App y ProductoCard?

App es el componente principal contenedor que gestiona el estado global de la y las llamadas a los servicios, mientras que ProductoCard es un componente secundario y reutilizable diseñado para mostrar la información visual de un producto mediante *props*

3. ¿Para qué utilizamos props?

Para transferir datos y funciones desde un componente padre hacia un componente hijo de manera unidireccional

4. ¿Qué función cumple map()?

Es un método de los arreglos en JavaScript que recorre una colección de datos (como la lista de productos) y transforma cada elemento en un componente visual JSX para renderizar listas dinámicas en pantalla

5. ¿Por qué utilizamos key={producto.id}?

Para que React pueda identificar de forma única cada elemento dentro de una lista renderizada, optimizando el rendimiento al actualizar, insertar o eliminar elementos del DOM virtual

Parte 3: Instalación de Axios y conexión del frontend con el backend

Preparar la comunicación entre **React** y el **backend Express** que desarrollamos anteriormente.

Instalar **Axios,** configuraremos **CORS** y comprobaremos la conexión con la API.

En este apartado mostraremos los productos del **backend** mediante **useState y useEffect.**

Configurar la comunicación entre una aplicación React y una API REST desarrollada con Express, utilizando Axios para realizar solicitudes HTTP y CORS para permitir el intercambio de recursos entre diferentes orígenes.

Conceptos fundamentales

¿Qué es Axios?

Es una biblioteca cliente HTTP basada en promesas

Podemos utilizarla para consumir los endpoints del backend:

Método	Axios	Operación
GET	axios.get()	Consultar productos
POST	axios.post()	Registrar productos
PUT	axios.put()	Actualizar productos
DELETE	axios.delete()	Eliminar productos

Ejemplo:

```
export default Encabezado;

const respuesta = await axios.get(
  "http://localhost:3000/api/productos"
);
```

La propiedad **respuesta.data** contiene los datos enviados por el backend.

Documentación oficial: Axios – Introducción y Axios – API.

https://axios.rest/pages/getting-started/first-steps

https://axios.rest/pages/advanced/api-reference

¿Qué es CORS?

es un mecanismo de seguridad implementado por los navegadores web

En nuestro proyecto tenemos dos servidores:

Aplicación	Dirección

Frontend React	http://localhost:5173
Backend Express	http://localhost:3000

Aunque ambos utilizan **localhost,** los puertos son diferentes. Por lo tanto, se consideran orígenes distintos.

Para permitir que el navegador acceda al backend, configuraremos CORS en Express.

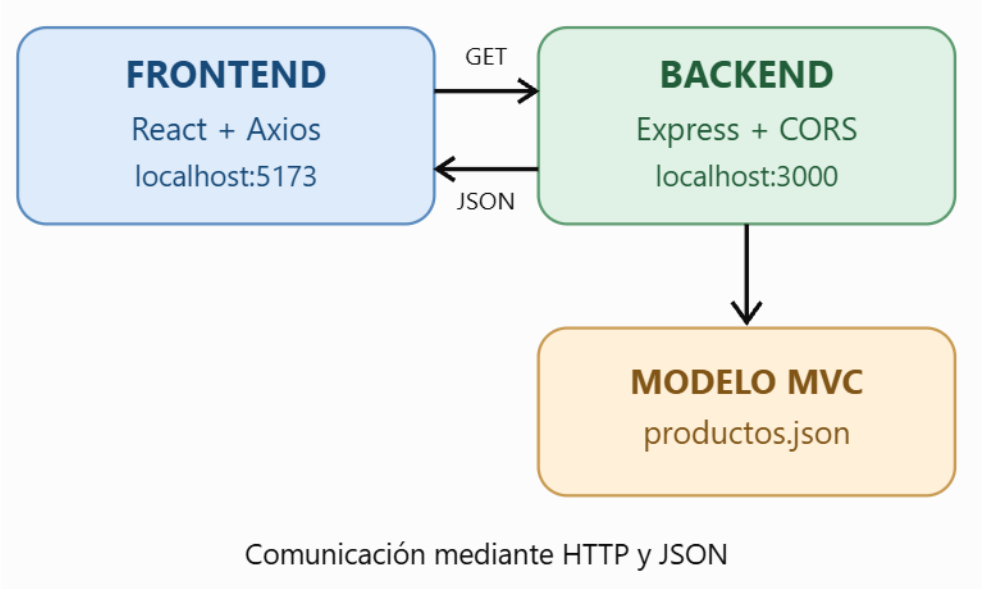
Documentación oficial: MDN – CORS y Express – Middleware CORS.

Intercambio de recursos de origen cruzado (CORS) - HTTP | MDN

https://developer.mozilla.org/es/docs/Web/HTTP/Guides/CORS

https://expressjs.com/en/resources/middleware/cors/

Arquitectura de comunicación



Comunicación mediante HTTP y JSON

Instalar Axios en el frontend

Abrir una terminal de Visual Studio Code y ubicarse en el proyecto **frontend-ventas.**

Ejecutar:

npm install axios

- **npm install** instala una dependencia.
- **axios** es el paquete que utilizaremos para realizar solicitudes HTTP.

- La dependencia se agrega a **package.json.**

Para comprobar la instalación:

npm list axios

Debe mostrarse la versión de Axios instalada.

```
frontend-ventas-mvc@0.0.0 C:\Aplicaciones-web-Lorena\sistema-ventas-mvc\frontend
-ventas-mvc
└── axios@1.21.0
```

Documentación oficial: Axios – Instalación.

https://axios.rest/pages/getting-started/first-steps

Instalar CORS en el backend

Abrir otra terminal y ubicarse en el proyecto **backend-ventas-mvc.**

Ejecutar:

npm install cors

Importante: Axios se instala en el frontend y CORS se configura en el backend.

Configurar CORS en Express

Abrir el archivo **src/app.js** del backend.

Actualmente tenemos la configuración de Express y las rutas de productos. Vamos a agregar el middleware **cors.**

```
import express from 'express';
import cors from "cors";
import productoRoutes from './routes/productoRoutes.js';

const app = express();
// Permitir solicitudes desde nuestro frontend
app.use(cors({
  origin: "http://localhost:5173"
}));

// Configurar el middleware JSON, interpretar cuerpos de json
app.use(express.json());

app.get('/', (req, res) => {
  res.send("API Sistema de Ventas MVC");
});

// Registrar las rutas
app.use("/api/productos", productoRoutes);

export default app;
```

Instrucción	Función
import cors from "cors"	Importa la libreria en el archivo de Node.js/Express para poder utilizar sus herramientas de control de acceso
app.use(cors(...))	Activa y aplica el middleware de CORS de forma global
origin	Es la propiedad de configuración donde especificamos con precisión exacta qué dominios o direcciones web tienen permitido hacer peticiones al servidor
http://localhost:5173	Es la dirección (origen) típica de una app de React en desarrollo (usando Vite)

Documentación oficial: Express – Configuración de CORS.

https://expressjs.com/en/resources/middleware/cors/

Ejecutar el backend

En la terminal del backend:

npm run dev

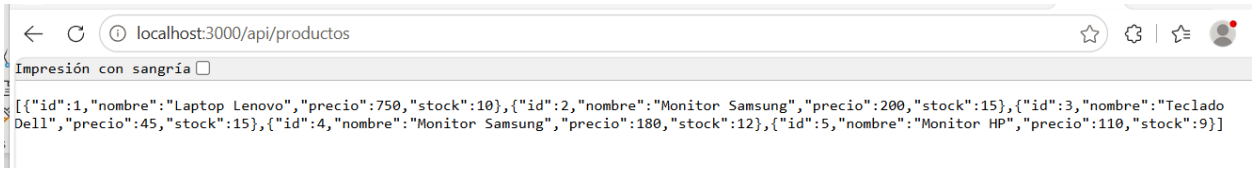
Verificar que aparezca:

Servidor ejecutándose en **http://localhost:3000**

Abrir en el navegador:

http://localhost:3000/api/productos

Debe mostrarse el arreglo JSON de productos.



¿Por qué debemos ejecutar el backend antes de realizar solicitudes desde React?

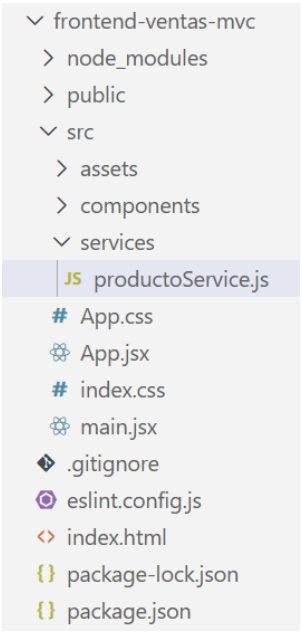
porque el backend es el que provee las rutas, la lógica de negocio y el acceso a la base de datos; si el servidor no está encendido, no hay nadie escuchando las peticiones y las llamadas de React fallarán instantáneamente

Crear un archivo para configurar Axios

En el frontend, dentro de **src,** crear una carpeta llamada **services.**

Dentro de ella crear el archivo **productoService.js.**

La estructura quedará así:



¿Por qué creamos una carpeta services?

para separar las peticiones de red y la comunicación con APIs externas de la lógica visual de los componentes

Configurar Axios

Abrir **src/services/productoService.js** y escribir:

```
1   import axios from "axios";
2
3   const api = axios.create({
4     baseURL: "http://localhost:3000/api"
5   });
6
7   export const obtenerProductos = async () => {
8     const respuesta = await api.get("/productos");
9     return respuesta.data;
10  };
```

Integrar el token en Axios

Envío del token de acceso desde React

En el backend, la ruta para obtener productos está protegida mediante el middleware **verificarToken.**

```
router.get("/", verificarToken, obtenerProductos);
```

El middleware espera recibir el token en la cabecera **x-token,** pero actualmente Axios no lo envía.

Por lo tanto, vamos a modificar únicamente el archivo, utilizamos la propiedad **headers** dentro de **axios.create().**

```
1   import axios from "axios";
2
3   const api = axios.create({
4     baseURL: "http://localhost:3000/api",
5     headers:{
6       "x-token": "ventas2026"
7     }
8   });
9
10  export const obtenerProductos = async () => {
11    const respuesta = await api.get("/productos");
12    return respuesta.data;
13  };
```

Explicación del código

Instrucción	Función
import axios from "axios"	Importa la biblioteca Axios para poder realizar solicitudes HTTP desde el frontend hacia el servidor
axios.create()	Crea una instancia personalizada de Axios con configuraciones globales predefinidas
baseURL	Define la dirección URL fija principal del servidor backend, evitando repetir la ruta completa en cada petición
headers:	Permite enviar metadatos o información adicional
x-token:	Nombre de la cabecera personalizada utilizada para enviar el token de autenticación hacia el backend
ventas2026:	Es el valor o clave secreta del token de seguridad asignado para autorizar las solicitudes en este taller
async	Palabra reservada que indica que una función maneja código asíncrono y permite el uso de operaciones con promesas
await	Pausa la ejecución de la función asíncrona hasta que la promesa
api.get("/productos")	Envía una solicitud HTTP GET utilizando la instancia configurada para consultar y obtener la lista de productos del backend
respuesta.data	Contiene el cuerpo de la respuesta enviada por el servidor, extrayendo directamente los datos limpios en formato JSON
export	Permite exportar las funciones del servicio para que puedan ser importadas y utilizadas en otros componentes de React

La URL completa que Axios utilizará es:

http://localhost:3000/api/productos

Documentación oficial: Axios – Instancias.

https://axios.rest/pages/advanced/create-an-instance

Comprobar la conexión desde React

Realizar una prueba sencilla desde **App.jsx** utilizando un botón. Así se comprueba la conexión antes de implementar los **hooks** de **React.**

Comprobar que React puede solicitar los productos al backend mediante Axios y recibir la respuesta.

Modificar **src/App.jsx** por:

```jsx
import Encabezado from "./components/Encabezado";
import { obtenerProductos } from "./services/productoService";
import "./App.css";

function App() {

  const probarConexion = async () => {
    try {
      const productos = await obtenerProductos();
      console.log("Productos del backend:", productos);
      alert("Conexión exitosa. Revisar la consola.");
    } catch (error) {
      console.error("Error de conexión:", error);
      alert("No se pudo conectar con el backend.");
    }
  };

  return (
    <div className="contenedor">
      <Encabezado />

      <h2>Conexión con la API REST</h2>

      <button onClick={probarConexion}>
        Consultar productos
      </button>
    </div>
  );
}

export default App;
```

Documentación oficial: React – Responding to Events.

https://react.dev/learn/responding-to-events

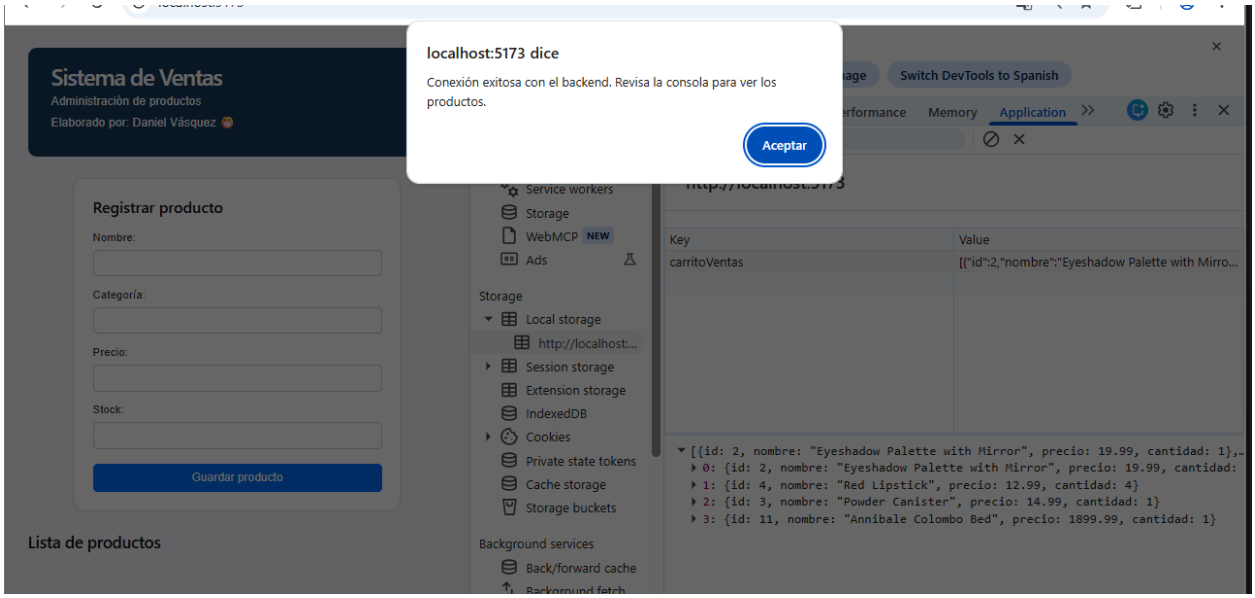
Ejecutar y comprobar

Debemos tener dos terminales activas:

Terminal	Puerto	Comando
Backend Express	3000	**npm run dev**
Frontend React	5173	**npm run dev**

Abrir **http://localhost:5173**, presionar el botón Consultar productos y revisar la consola del navegador con F12, en la pestaña Console.

Los productos mostrados dependerán de los registros que actualmente existan en **productos.json.** El resultado esperado es:



Importante: Se demuestra cómo el frontend envía el token al backend mediante Axios.

Actividad 1

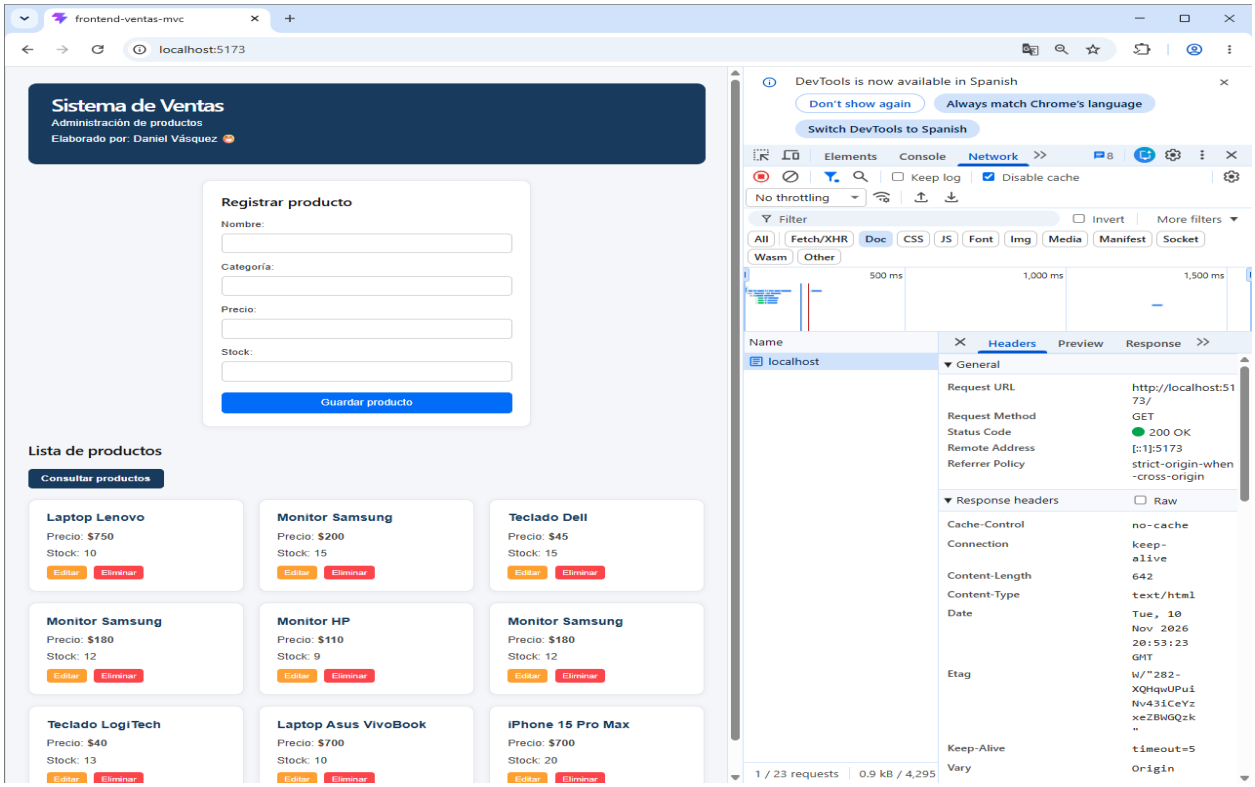
Verificar que Axios esté instalado correctamente utilizando npm list axios.

```
C:\Users\APLICACIONES WEB\Downloads\Semana-7\backend-ventas-mvc>npm list axios
sistema-ventas-mvc@1.0.0 C:\Users\APLICACIONES WEB\Downloads\Semana-7\backend-v
entas-mvc
└── axios@1.21.1IONES WEB\Downloads\Semana-7\backend-ventas-mvc>npm list axios
sistema-ventas-mvc@1.0.0 C:\Users\APLICACIONES WEB\Downloads\Semana-7\backend-v

C:\Users\APLICACIONES WEB\Downloads\Semana-7\backend-ventas-mvc>
```

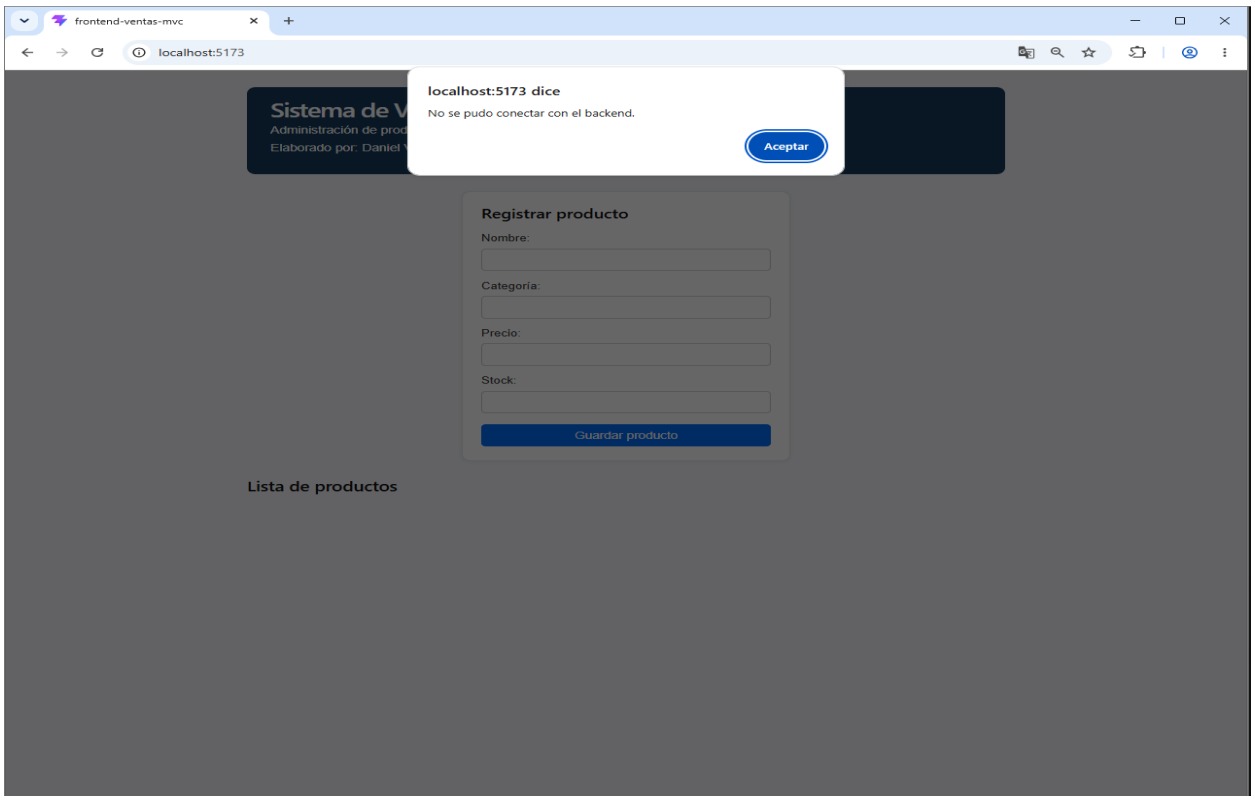
Actividad 2 : Comprobar la solicitud en Network

Abrir las herramientas de desarrollador del navegador, seleccionar la pestaña Network y comprobar que al presionar el botón se realiza una solicitud GET a /api/productos con código HTTP 200.



Actividad 3: Comprobar qué ocurre cuando el backend no está disponible

Detener temporalmente el servidor Express y volver a presionar el botón. Observar el error que aparece y explicar por qué ocurre. Después, volver a ejecutar el backend.



Preguntas

1. ¿Qué es Axios y para qué lo utilizamos?

Es una biblioteca cliente HTTP basada en promesas que nos permite realizar peticiones asíncronas (GET, POST, PUT, DELETE) desde el navegador hacia el servidor backend

2. ¿Por qué necesitamos configurar CORS en Express?

Porque por políticas de seguridad del navegador, las aplicaciones que corren en diferentes orígenes o puertos bloquean la comunicación a menos que el servidor autorice explícitamente el acceso mediante CORS

3. ¿Qué función cumple axios.create()?

Permite instanciar un cliente HTTP personalizado de Axios con configuraciones predeterminadas compartidas, como la URL base y las cabeceras de autorización

4. ¿Para qué sirve la propiedad baseURL?

establece la dirección raíz fija de la API del servidor, evitando tener que escribir la URL completa en cada solicitud HTTP individual

5. ¿Qué contiene respuesta.data?

Contiene los datos puros enviados como respuesta por el servidor backend en formato JSON

6. ¿Por qué utilizamos try y catch?

Para manejar de manera controlada los errores asíncronos

Parte 4. Consultar y mostrar productos utilizando React

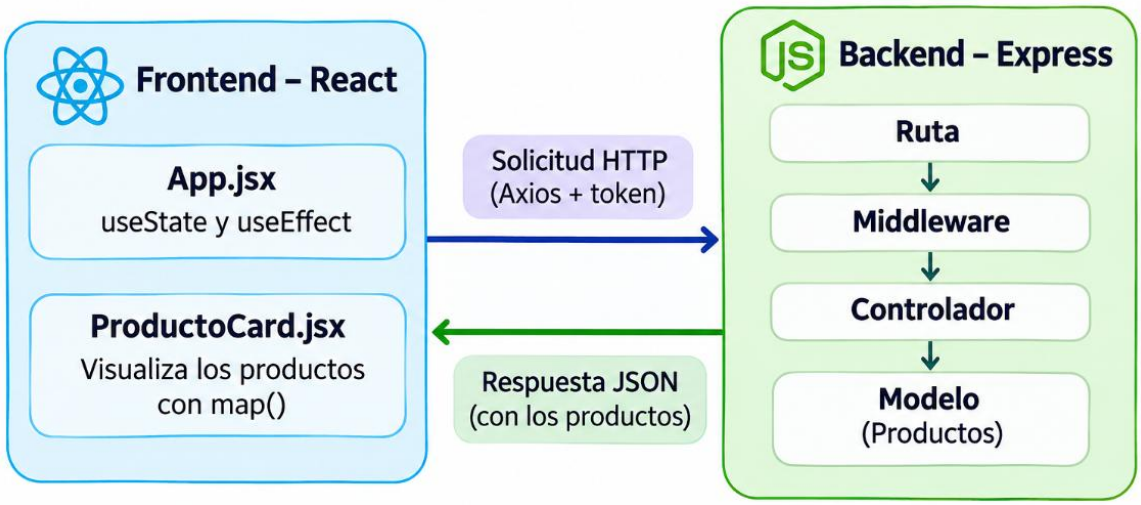
Consultar los productos almacenados en el backend del Sistema de Ventas y mostrarlos en la interfaz de React mediante los componentes que desarrollamos anteriormente.

Para ello utilizaremos:

- **useState:** Es una función especial (Hook) de React que permite a un componente recordar y actualizar sus propios datos
- **useEffect:** Es un Hook de React que se usa para ejecutar código de forma automática en momentos específicos del ciclo de vida de un componente
- **Axios:** Es una librería externa que se utiliza para realizar peticiones HTTP

- **map():** Es un método nativo de JavaScript que recorre un arreglo (lista) de elementos y transforma cada uno de ellos para crear una nueva lista
- **ProductoCard:** Es un componente visual personalizado creado por el desarrollador para representar el diseño de un producto individual

Arquitectura de comunicación REACT + EXPRESS



Modificar App.jsx

Incorporamos los hooks de React y el componente **ProductoCard.**

Reemplazar el contenido de src/App.jsx por:

```jsx
import { useState, useEffect } from "react";
import Encabezado from "./components/Encabezado";
import ProductoCard from "./components/ProductoCard";
import { obtenerProductos } from "./services/productoService";
import "./App.css";

function App() {

  const [productos, setProductos] = useState([]);

  useEffect(() => {

    const cargarProductos = async () => {
      try {
        const datos = await obtenerProductos();
        setProductos(datos);
      } catch (error) {
        console.error("Error al cargar productos:", error);
      }
    };

    cargarProductos();

  }, []);

  return (
    <div className="contenedor">
      <Encabezado />

      <h2>Lista de productos</h2>

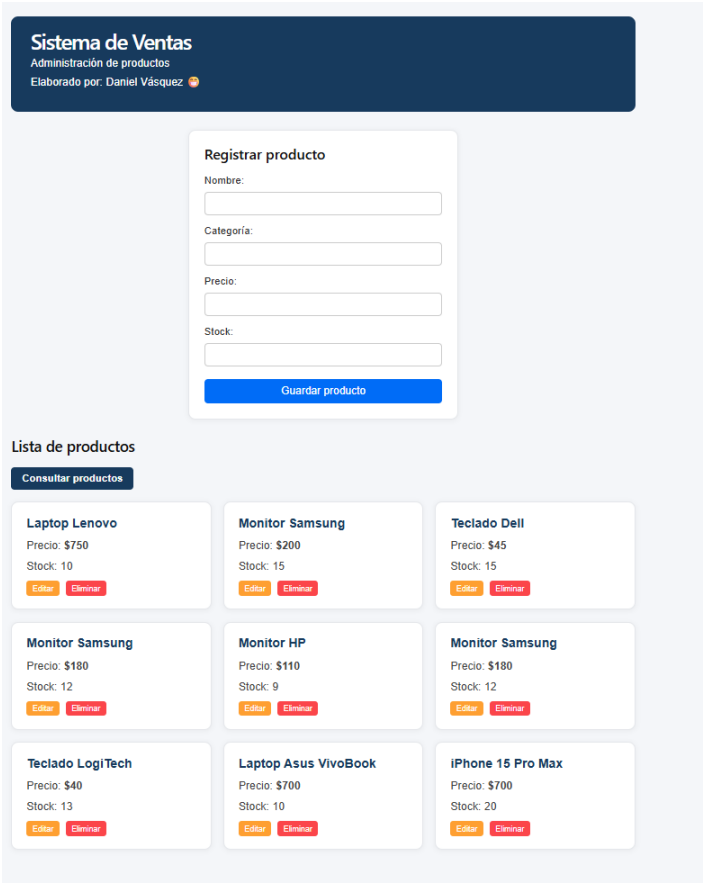
      <div className="productos">
        {productos.map((producto) => (
          <ProductoCard
            key={producto.id}
            nombre={producto.nombre}
            precio={producto.precio}
            stock={producto.stock}
          />
        ))}
      </div>
    </div>
  );
}

export default App;
```

Ejecutar y comprobar

Mantén ejecutándose el backend y el frontend con npm run dev.

Abre: http://localhost:5173



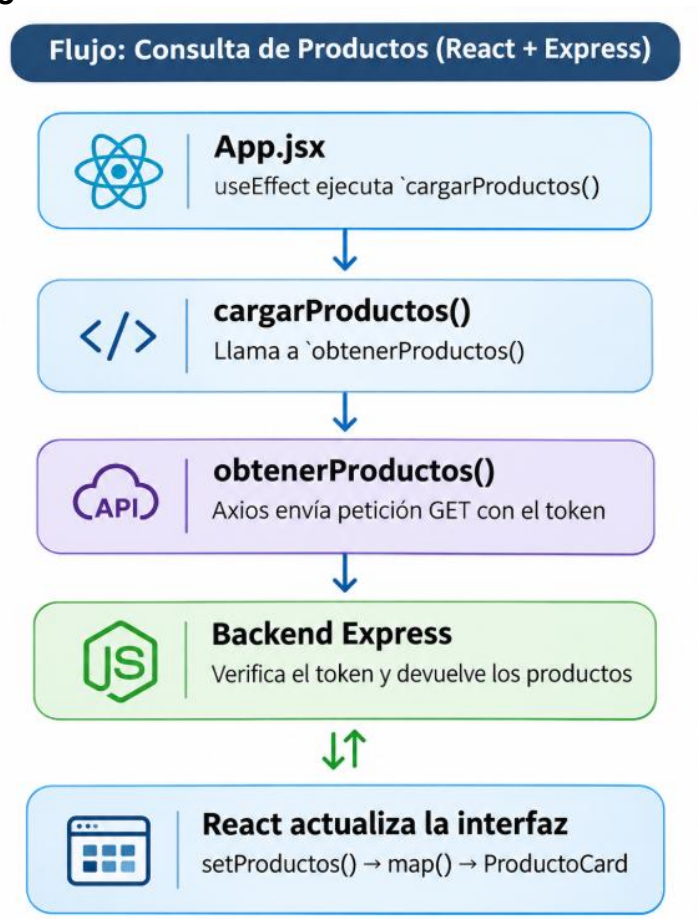
Recomendación: mantener separadas las responsabilidades del servicio y del componente React, utilizando async/await y try/catch, como ya comenzamos a enseñar en la Parte 3.

La organización será la siguiente:

Archivo	Responsabilidad	URL
productoService.js	Gestiona la comunicación con la API	https://axios.rest/pages/advanced/create-an-instance
App.jsx	Actúa como el orquestador principal de la app	https://react.dev/reference/react/useState

ProductoCard.jsx	Renderiza un producto individual	
Encabezado.jsx	Muestra la barra de navegación o cabecera global	

¿Cómo se relacionan estas funciones?



Instrucción	Función
productos.map()	Recorre la lista de productos
producto	Es la variable temporal (iterador)
<ProductoCard />	Es el componente visual
key={producto.id}	Es un identificador único obligatorio para React
nombre, precio, stock	Son las propiedades (props) o datos específicos

Renderizar listas

https://react.dev/learn/rendering-lists

Passing Props to a Component

https://react.dev/learn/passing-props-to-a-component

Parte 5. Registrar productos utilizando React y Axios

Implementar el registro de nuevos productos desde el frontend utilizando un formulario React y una solicitud HTTP **POST** al backend Express.

Al finalizar esta parte, podrás ingresar los datos de un producto, enviarlos al backend y actualizar la lista de productos sin recargar la página.



Incorporar la función registrarProducto() en el servicio

En el archivo: **productoService.js**

Agregar la siguiente función al final del archivo:

```
export const registrarProducto = async (producto) => {
  const respuesta = await api.post("/productos", producto);
  return respuesta.data;
};
```

La URL completa de la solicitud será:

http://localhost:3000/api/productos

Documentación oficial: Axios – API y métodos HTTP.

https://axios.rest/pages/advanced/api-reference

La función registrarProducto() reutiliza la instancia api, que ya contiene la dirección del backend y el token ventas2026.

Registrar productos utilizando React y Axios.

Crear el formulario para ingresar nuevos productos.

Crear el componente FormularioProducto.jsx

Ahora crearemos **FormularioProducto.jsx,** cuya responsabilidad será capturar los datos ingresados por el usuario y enviarlos al componente principal para su registro.

De esta manera, evitamos colocar toda la lógica de la interfaz dentro de **App.jsx.**

Documentación oficial: React – Creación de componentes.

https://react.dev/learn/your-first-component

Documentación oficial: React – Componente input.

https://react.dev/reference/react-dom/components/input

Crear el archivo FormularioProducto.jsx

Dentro de **src/components,** crea el archivo:

FormularioProducto.jsx

```jsx
import { useState } from "react";

function FormularioProducto({ onRegistrar }) {

  const [nombre, setNombre] = useState("");
  const [precio, setPrecio] = useState("");
  const [stock, setStock] = useState("");

  const manejarEnvio = (e) => {
    e.preventDefault();

    const nuevoProducto = {
      nombre: nombre.trim(),
      precio: Number(precio),
      stock: Number(stock)
    };

    onRegistrar(nuevoProducto);

  };
```

```jsx
  return (
    <form className="formulario-producto" onSubmit={manejarEnvio}>
      <h2>Registrar producto</h2>

      <div>
        <label>Nombre:</label>
        <input
          type="text"
          value={nombre}
          onChange={(e) => setNombre(e.target.value)}
          required
        />
      </div>

      <div>
        <label>Precio:</label>
        <input
          type="number"
          value={precio}
          onChange={(e) => setPrecio(e.target.value)}
          min="0.01"
          step="0.01"
          required
        />
      </div>

```

```
47
48          <div>
49            <label>Stock:</label>
50            <input
51              type="number"
52              value={stock}
53              onChange={(e) => setStock(e.target.value)}
54              min="0"
55              step="1"
56              required
57            />
58          </div>
59
60          <button type="submit">
61            Guardar producto
62          </button>
63        </form>
64      );
65    }
66
67    export default FormularioProducto;
```

¿Qué hace manejarEnvio()?

procesa y valida los datos de un formulario cuando el usuario hace clic en el botón de enviar o presiona Enter

Código	Descripción
e.preventDefault()	Evita que la página web se recargue automáticamente
nombre.trim()	Elimina todos los espacios en blanco sobrantes
Number()	Transforma un texto (String) en un número rea
nuevoProducto	Es el nuevo objeto creado en memoria
onRegistrar(nuevoProducto)	Es la función que envía el nuevo producto hacia el componente padre

¿Por qué no llamamos directamente a Axios desde el formulario?

Porque queremos mantener separadas las responsabilidades:

Archivo	Responsabilidad
FormularioProducto.jsx	Captura y valida los datos del usuario
App.jsx	Orquesta el estado y distribuye la información.

productoService.js	Guarda el producto en el servidor

Documentación oficial: React – Compartir estado entre componentes.

https://react.dev/learn/sharing-state-between-components

Conectarlo con **App.jsx** para que los datos ingresados se envíen al backend y se actualice la lista de productos.

Integrar FormularioProducto en App.jsx

Incorporaremos el formulario para registrar productos.

Para ello, necesitamos una función llamada **guardarProducto()** en **App.jsx.**

¿Por qué necesitamos guardarProducto()?

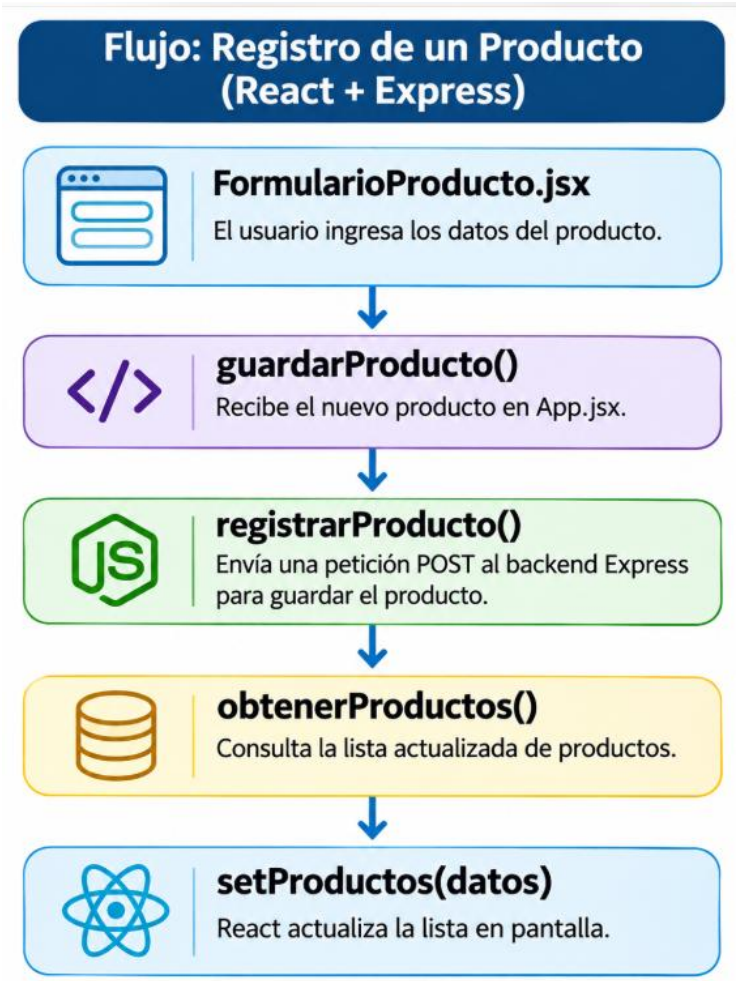
Recordemos que en **FormularioProducto.jsx** utilizamos:

```
import { useState, useEffect } from "react";
import Encabezado from "./components/Encabezado";
import ProductoCard from "./components/ProductoCard";
import FormularioProducto from "./components/FormularioProducto";
import {
  obtenerProductos,
  registrarProducto
} from "./services/productoService";
import "./App.css";

function App() {
```

```
30    // Registrar un producto y actualizar la lista
31    const guardarProducto = async (nuevoProducto) => {
32      try {
33        await registrarProducto(nuevoProducto);
34
35        const datos = await obtenerProductos();
36        setProductos(datos);
37
38      } catch (error) {
39        console.error("Error al registrar producto:", error);
40      }
41    };
42
```

```
43    return (
44      <div className="contenedor">
45        <Encabezado />
46        <h2>Registrar producto</h2>
47
48        <FormularioProducto onRegistrar={guardarProducto} />
49
50        <h2>Lista de productos</h2>
```



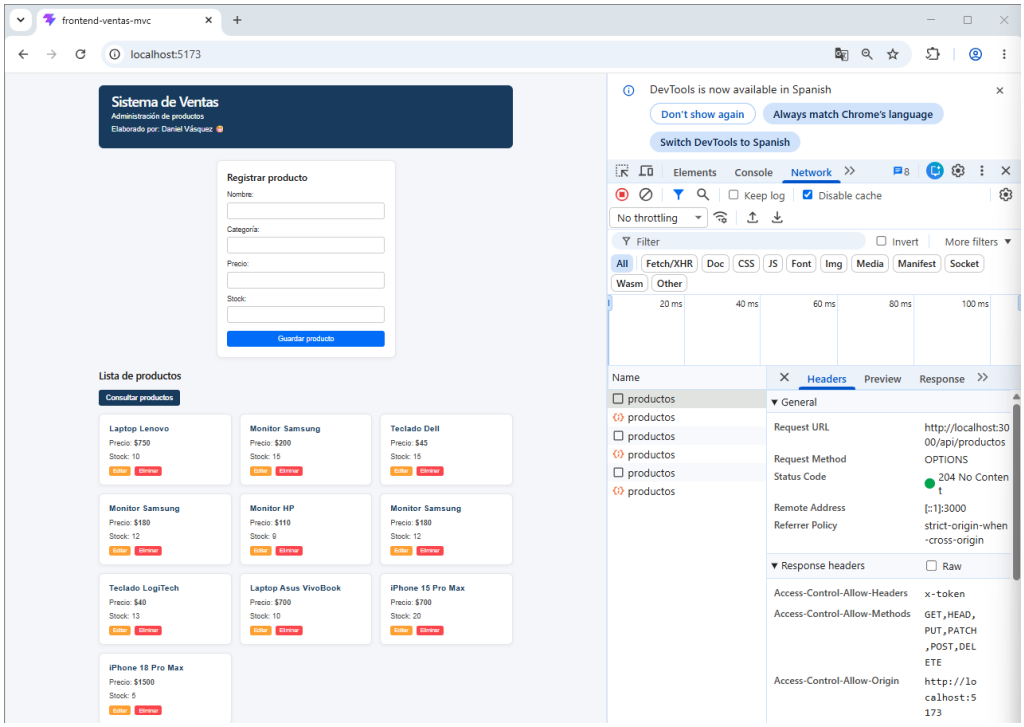
Agregar los estilos al foriulario al final de App.cs. Descárgate de OneDrive

Verificar FormularioProducto.jsx

Para que los estilos se apliquen, la etiqueta **<form>** debe tener esta clase:

```
21    return (
22      <form className="formulario-producto" onSubmit={manejarEnvio}>
23        <h2>Registrar producto</h2>
```

Verificar



También revisa el archivo json

```
    "id": 7,
    "nombre": "Parlantes",
    "precio": 5,
    "stock": 12
  },
  {
    "id": 8,
    "nombre": "Micrófono",
    "precio": 7,
    "stock": 20
  }
]
```

Limpiar el formulario

En FormularioProducto.jsx, busca la función manejarEnvio y agrega la limpieza de los campos

```
9   const manejarEnvio = async (e) => {
10    e.preventDefault();
11
12    const nuevoProducto = {
13      nombre: nombre.trim(),
14      precio: Number(precio),
15      stock: Number(stock)
16    };
17
18    await onRegistrar(nuevoProducto);
19
20    // Limpiar el formulario
21    setNombre("");
22    setPrecio("");
23    setStock("");
24  };
```

¿Qué hace cada instrucción?

Instrucción	Función
await onRegistrar(nuevoProducto)	Espera a que el producto se guarde con éxito
setNombre("")	Limpia el campo de texto del nombre
setPrecio("")	Borra el valor del campo de precio
setStock("")	Limpia el campo numérico del stock

Limpiar el formulario después de registrar un producto

Actualmente, cuando ingresamos un producto y presionamos Guardar producto, los datos se envían al backend, pero los campos permanecen con los valores ingresados.

Vamos a mejorar este comportamiento para que, después de registrar correctamente un producto, el formulario quede vacío y permita ingresar otro.

Modificar el archivo:

FormularioProducto.jsx

La función **onRegistrar()** ejecuta **guardarProducto()** en **App.jsx.**

Sin embargo, como el registro es una operación asíncrona, debemos esperar a que termine antes de limpiar los campos.

Modificar guardarProducto() en App.jsx

Reemplaza únicamente la función **guardarProducto()** por:

```
// Registrar un producto y actualizar la lista
const guardarProducto = async (nuevoProducto) => {
  try {
    await registrarProducto(nuevoProducto);

    const datos = await obtenerProductos();
    setProductos(datos);
    return true;

  } catch (error) {
    console.error("Error al registrar producto:", error);
    return false;
  }
};
```

Modificar manejarEnvio() en FormularioProducto.jsx

Ahora reemplaza únicamente la función **manejarEnvio()** por:

```
9   const manejarEnvio = async (e) => {
10    e.preventDefault();
11
12    const nuevoProducto = {
13      nombre: nombre.trim(),
14      precio: Number(precio),
15      stock: Number(stock)
16    };
17
18    const registrado = await onRegistrar(nuevoProducto);
19
20    if (registrado) {
21      setNombre("");
22      setPrecio("");
23      setStock("");
24    }
25  };
```

Explicación del código

Instrucción	Función
async	Indica que una función es asíncrona
await onRegistrar(nuevoProducto)	Pausa la ejecución del código y espera
registrado	Es una variable booleana
if (registrado)	Evalúa si la operación fue exitosa
setNombre("")	Limpia el campo de texto del nombre
setPrecio("")	Borra el valor introducido en el campo de precio
setStock("")	Limpia el campo numérico del stock

Documentación oficial: **React – useState.**

https://react.dev/reference/react/useState

Comprobar el funcionamiento

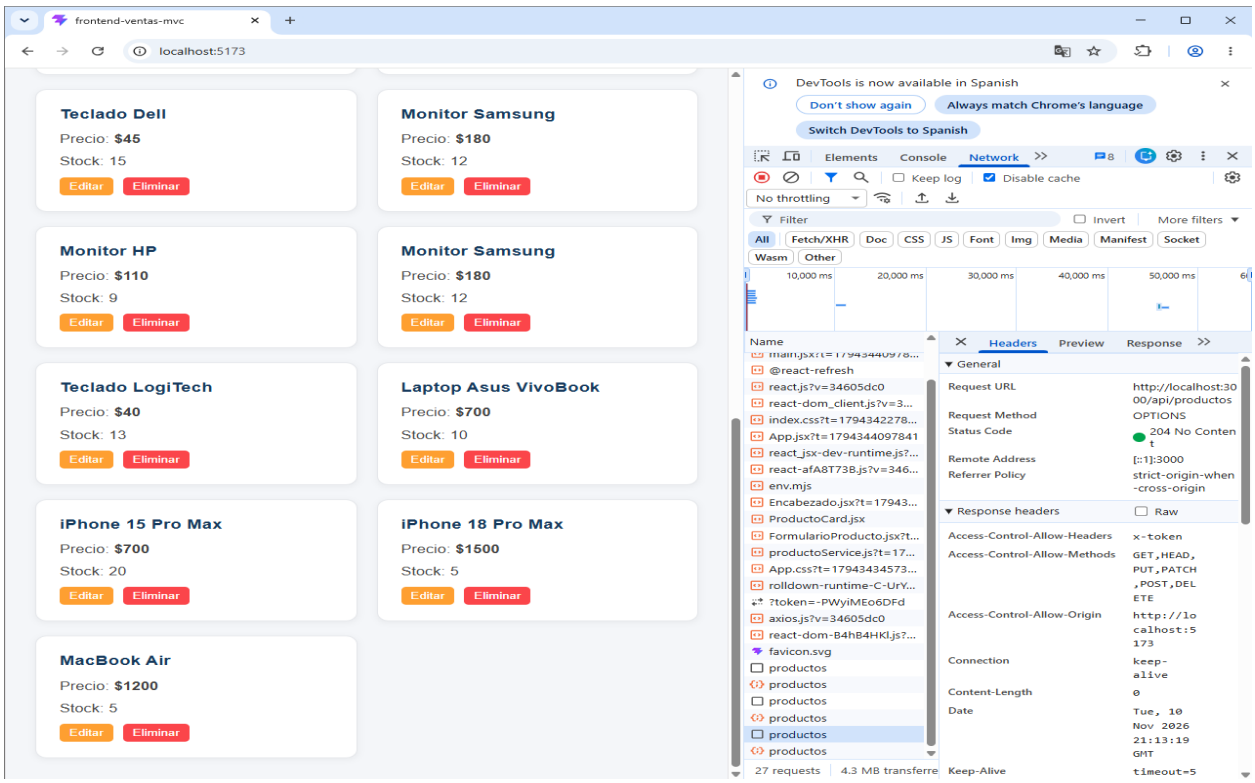
Ejecuta la aplicación y registra un producto de prueba.

Al presionar Guardar producto, comprueba que:

1. El producto se registre correctamente en el backend.



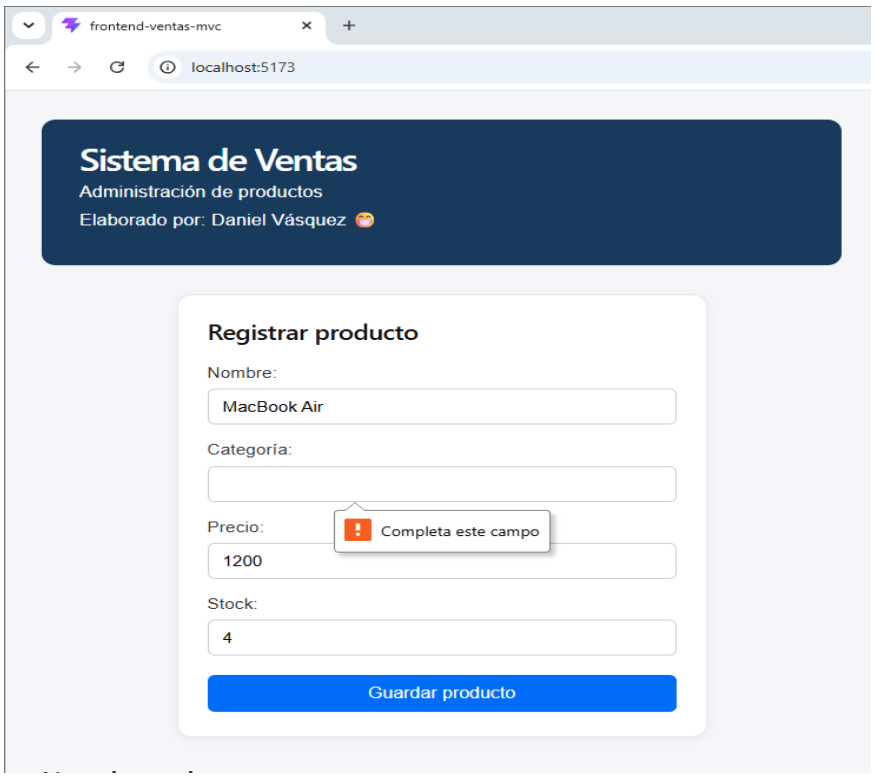
2. La lista de productos se actualice automáticamente.



3. Los tres campos del formulario queden vacíos.



4. Si ocurre un error, los valores ingresados permanezcan en el formulario.



Parte 6. Modificar productos utilizando React y Axios (PUT)

Implementar la modificación de productos desde el frontend utilizando React, Axios y el método HTTP **PUT,** permitiendo actualizar los datos de un producto registrado en el backend del Sistema de Ventas.

Al finalizar podrás seleccionar un producto, modificar sus datos y visualizar los cambios sin recargar la página.



Función	Archivo	Responsabilidad
obtenerProductos()	productoService.js	Realiza una petición HTTP GET al backend
registrarProducto()	productoService.js	

actualizarProducto()	productoService.js	Realiza una petición HTTP POST al backend
seleccionarProducto()	App.jsx	Realiza una petición HTTP PUT o PATCH al backend
guardarCambios()	App.jsx	Guarda el producto que se desea editar en un estado centra

Incorporar actualizarProducto() en el servicio

Necesitamos una función que permita modificar los datos de un producto existente mediante **PUT.**

Esta función se ubicará en el mismo archivo productoService.js, porque su responsabilidad también es comunicarse con el backend.

Archivo: **productoService.js**

Agrega al final del archivo:

```
export const actualizarProducto = async (id, producto) => {
  const respuesta = await api.put(`/productos/${id}`, producto);
  return respuesta.data;
};
```

Explicación del código

Instrucción	Función
actualizarProducto	Es el nombre de la función asíncrona
id	Es el primer parámetro que recibe la función
producto	Es el segundo parámetro (un objeto) que contiene los nuevos datos modificados
api.put()	Ejecuta una petición HTTP PUT
`/productos/${id}`	Es la URL dinámica del endpoint
respuesta.data	Extrae el cuerpo del mensaje de éxito enviado por el servidor
export	Permite que esta función sea pública

¿Qué significa ${id}?

Es una interpolación o plantilla de cadena en JavaScript que inserta dinámicamente el valor numérico o de texto de la variable id dentro de la ruta de la petición

Documentación oficial: Axios – API y métodos HTTP.

https://axios.rest/pages/advanced/api-reference

Con esta incorporación, nuestro servicio dispone de tres operaciones:

Método	Función	Operación
GET	obtenerProductos()	Consultar productos.
POST	registrarProducto()	Registrar productos.
PUT	actualizarProducto()	Modificar productos.

Agregar el botón Editar en ProductoCard.jsx

Incorporar el botón Editar en **ProductoCard.jsx,** explicando cómo identificar el producto seleccionado y enviar esa información al componente principal mediante una propiedad **(prop).**

Documentación oficial: React – Manejo de eventos.

https://react.dev/learn/responding-to-events

Incorporar el botón Editar en cada tarjeta, permitirá identificar el producto seleccionado.

Utilizaremos una propiedad onEditar

En **App.js** en la Parte 5 utilizamos

```
<FormularioProducto onRegistrar={guardarProducto} />
```

La propiedad **onRegistrar** permite que el componente hijo ejecute una función definida en el componente padre.

Aplicaremos el mismo principio con **ProductoCard:**

```
<ProductoCard onEditar={seleccionarProducto} />
```

Así, cada tarjeta podrá comunicar a App.jsx qué producto se desea modificar.

Documentación oficial: React – Responder a eventos.

https://react.dev/learn/responding-to-events

En el archivo: **ProductoCard.jsx**

El componente quedará así:

```jsx
function ProductoCard({ id, nombre, precio, stock, onEditar }) {

  return (
    <article className="producto-card">
      <h3>{nombre}</h3>
      <p>Precio: ${precio}</p>
      <p>Stock: {stock}</p>
      <button onClick={() => onEditar(id)}>
        Editar
      </button>
    </article>
  );
}

export default ProductoCard;
```

Explicación del código

Instrucción	Función
id	Es el identificador único del producto actual
onEditar	Es una función recibida a través de *props*
<button>	Es la etiqueta HTML nativa
onClick	Es el manejador de eventos de React
() => onEditar(id)	Es una función de flecha anónima

¿Qué sucede al presionar Editar?



Modificar App.jsx para implementar esa comunicación

Realiza únicamente el cambio en **ProductoCard.jsx.**

Crear el estado productoSeleccionado

Agrega:

```
const [productoSeleccionado, setProductoSeleccionado] = useState(null);
```

Elemento	Función
productoSeleccionado	Almacena el producto que deseamos modificar.
setProductoSeleccionado()	Actualiza el producto seleccionado.
useState(null)	Indica que inicialmente no existe ningún producto seleccionado.

Utilizamos null porque, al cargar la aplicación, el usuario todavía no ha seleccionado ningún producto.

Documentación oficial: React – useState.

https://react.dev/reference/react/useState

Crear la función seleccionarProducto()

Esta función recibirá el identificador enviado desde ProductoCard.jsx.

En App.jsx, después de guardarProducto(), agrega:

```
47  // Seleccionar un producto para modificar
48  const seleccionarProducto = (id) => {
49    const producto = productos.find(
50      (producto) => producto.id === id
51    );
52
53    setProductoSeleccionado(producto);
54
55    console.log("Producto seleccionado:", producto);
56  };
```

Explicación del código:

Instrucción	Función
seleccionarProducto(id)	Recibe el identificador del producto.
productos.find()	Busca un producto dentro del arreglo.
producto.id === id	Compara los identificadores.
setProductoSeleccionado(producto)	Guarda el producto seleccionado en el estado.
console.log()	Permite comprobar qué producto fue seleccionado.

¿Qué hace find()?

Recorre el arreglo de productos buscando y devolviendo el primer objeto que cumple estrictamente con la condición evaluada

Documentación oficial: MDN – Array.find().

https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/Array/find

Modificar ProductoCard dentro de App.jsx

En el return de App.jsx,

```
82        <div className="productos">
83          {productos.map((producto) => (
84            <ProductoCard
85              key={producto.id}
86              id={producto.id}
87              nombre={producto.nombre}
88              precio={producto.precio}
89              stock={producto.stock}
90              onEditar={seleccionarProducto}
91            />
92          ))}
93        </div>
94      </div>
95    );
96  }
```

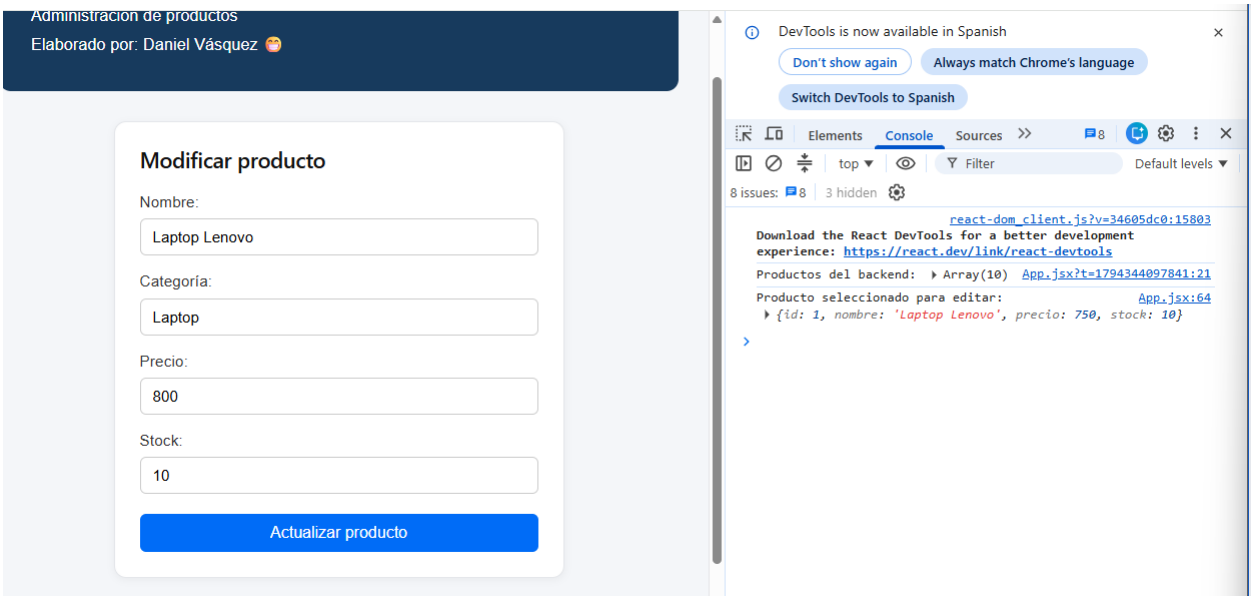
¿Cómo funciona la comunicación?



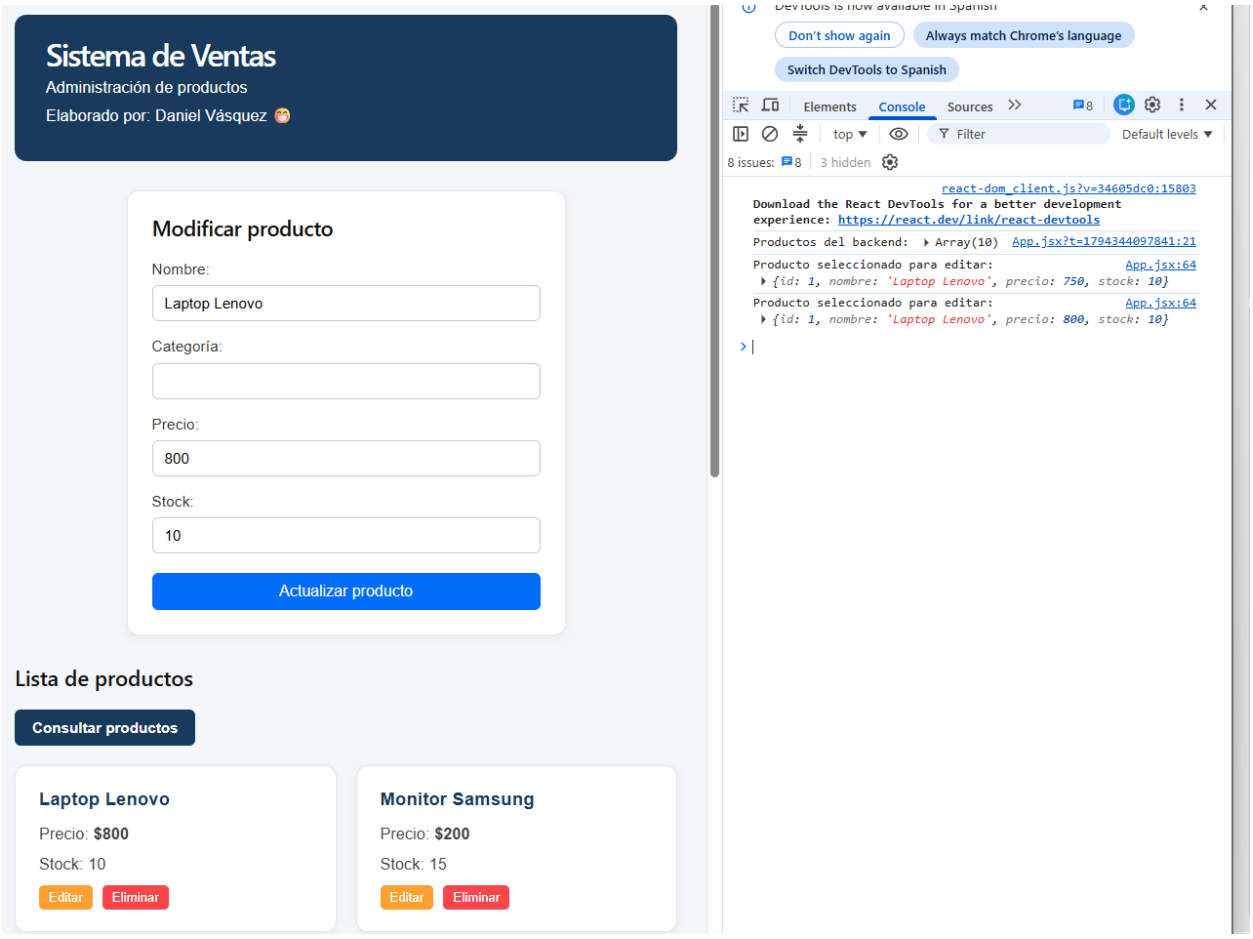
Comprobar el funcionamiento

Ejecuta tu aplicación en http://localhost:5173.

1. Abre la consola del navegador con F12.
2. Presiona el botón Editar de cualquier producto.
3. Comprueba que aparezca Producto seleccionado: con los datos correspondientes.



4. Selecciona otro producto y verifica que cambie la información mostrada en la consola.



Resultado esperado: React identifica correctamente el producto seleccionado y lo almacena en productoSeleccionado.

Si no hay respuesta, modificar temproalmente en App.jsx, reemplaza la función seleccionarProducto() por:

```
const seleccionarProducto = (id) => {

  console.log("ID recibido:", id);
  console.log("Productos disponibles:", productos);

  const producto = productos.find(
    (producto) => Number(producto.id) === Number(id)
  );

  setProductoSeleccionado(producto);

  console.log("Producto seleccionado:", producto);
};
```

Verificar ProductoCard.jsx



Actualmente, nuestro formulario utiliza manejarEnvio() para registrar productos mediante POST.

Ahora necesitamos que el mismo formulario permita realizar dos operaciones:

- Registrar: cuando no existe un producto seleccionado, utilizaremos POST.
- Actualizar: cuando existe un producto seleccionado, utilizaremos PUT.

No crearemos otro formulario. Reutilizaremos FormularioProducto.jsx.

. Funciones que intervienen

Función	Archivo	Responsabilidad
manejarEnvio()	FormularioProducto.jsx	Recibir los datos y determinar qué operación ejecutar.
guardarProducto()	App.jsx	Registrar un producto mediante POST.

guardarCambios()	App.jsx	Coordinar la modificación del producto.
actualizarProducto()	productoService.js	Enviar la solicitud PUT al backend.

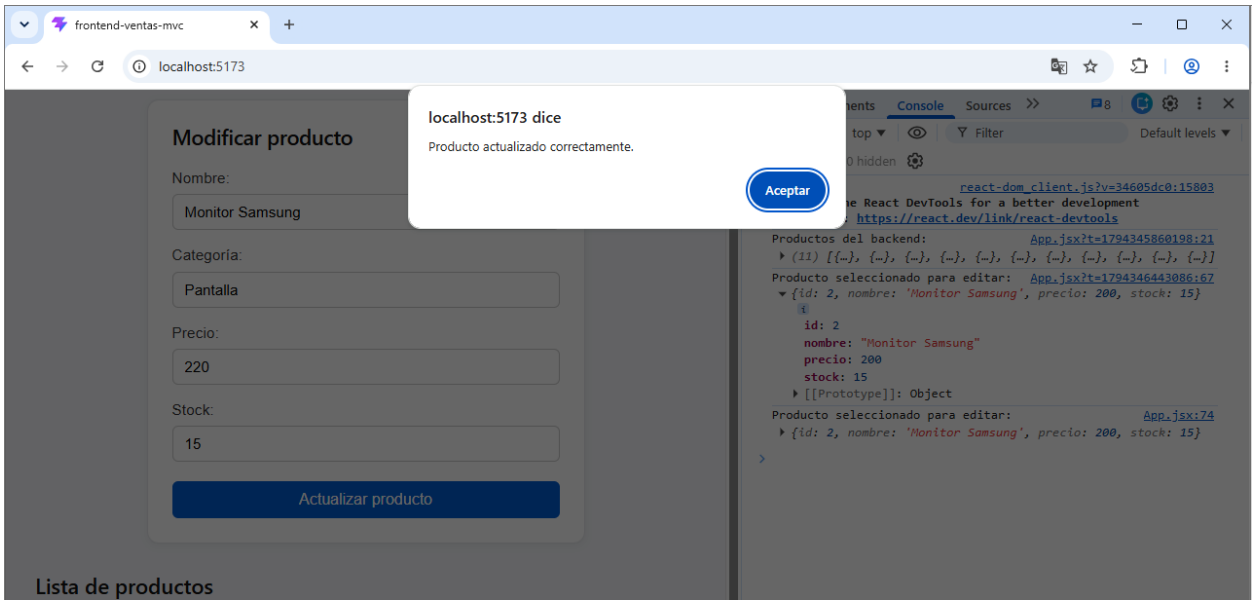
La función actualizarProducto() ya fue incorporada en el Paso 1.

Modificar App.jsx

Importar actualizarProducto()

```
import {
  obtenerProductos,
  registrarProducto,
  actualizarProducto,
} from "./services/productoService";
```

```
// UPDATE (PUT)
const editarProducto = async (productoData) => {
  await actualizarProducto(productoSeleccionado.id, productoData);
  setProductoSeleccionado(null);
  alert("Producto actualizado correctamente.");
};
```

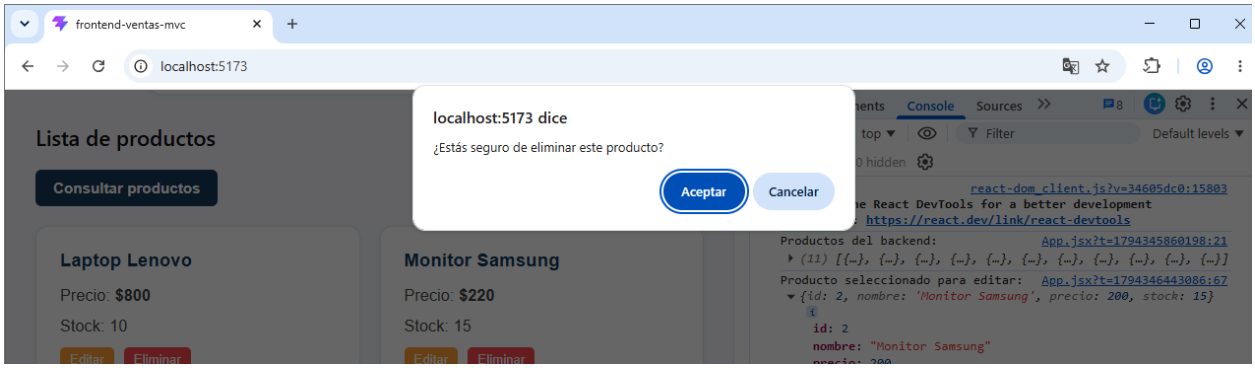


DESAFÍO

Realiza la función Eliminar

```
import {
  obtenerProductos,
  registrarProducto,
  actualizarProducto,
  eliminarProducto,
} from "./services/productoService";
import "./App.css";
```

```
// DELETE
const borrarProducto = async (id) => {
  if (!window.confirm("¿Estás seguro de eliminar este producto?")) return;
  try {
    await eliminarProducto(id);
    await refrescarProductos();
  } catch (error) {
    console.error("Error al eliminar:", error);
    alert("No se pudo eliminar el producto.");
  }
};
```



PRESENTACIÓN

Una vez culminada tu tarea, súbela en el apartado del aula virtual.

Recuerda el nombre del archivo deberá ser: **Taller_ApellidoNombre**

RECURSOS NECESARIOS

- Acceso a Internet.
- Navegador
- Visual Studio Code,
- React